

# Gestor de Recetas

Desarrolla una aplicación web llamada "Gestor de Recetas" donde los usuarios pueden registrarse, **iniciar sesión y gestionar sus recetas**. Cada usuario podrá crear, listar y ver sus recetas. La aplicación debe permitir la autenticación de usuarios y ofrecer una interfaz para la creación y visualización de recetas.

## Clases y Prototipos de Métodos

### 1. Clase Usuario

#### ○ Propiedades:

- int \$id
- string \$nombre\_usuario
- string \$email
- string \$contrasena

#### ○ Métodos:

- public function \_\_construct()
- public function mostrarPerfil()
- public static function registrar(string \$nombre\_usuario, string \$email, string \$contrasena)
- public static function validar()

### 2. Clase Receta

#### ○ Propiedades:

- int \$id
- string \$nombre\_receta
- string \$ingredientes
- int \$tiempo\_preparacion
- string \$dificultad
- int \$usuario\_id

#### ○ Métodos:

- public function \_\_construct()
- public function crearReceta()
- public static function obtenerRecetas(int \$usuario\_id)

### 3. Clase ControladorSesion

#### ○ Métodos:

- public function index()
- public function logout()
- public function login()

### 4. Clase ControladorRecetas

#### ○ Métodos:

- public function nuevaReceta()
- public function guardarReceta()
- public function mostrarRecetas(int \$usuario\_id)

### 5. Clase ControladorUsuario

#### ○ Métodos:

- public function index()
- public function nuevoUsuario()
- public function guardarUsuario()

**Debéis usar `FrontController` que se entrega con la aplicación**

## **Flujo de la Aplicación**

### **1. Petición Inicial:**

- Cuando se accede a `index.php` sin parámetros en la URL (ej., `http://localhost/rectas/index.php`), el `FrontController` llama a `manejarPetición()`, que a su vez llama a `cargarControladorDefecto()`.

### **2. Carga del Controlador:**

- `cargarControladorDefecto()` carga `SessionController` y llama a su método `index()`.

### **3. Decisión de Vista:**

- En el método `index()` de `SessionController`, se verifica si el usuario está logueado.
  - Si está logueado, se muestra el header, el perfil y el footer.
  - Si no está logueado, se muestra la vista login.

De esta manera, el `SessionController` actúa como el punto de partida para gestionar el flujo de usuario en función de su estado de sesión, garantizando que los usuarios vean las vistas adecuadas según su autenticación.

Debéis implementar las clases y sus métodos correspondientes según esta estructura, asegúrate de manejar la lógica de negocio dentro de los controladores y mantener la vista y el modelo separados.